

Brain Health Data Notes

Data Acquisition and Processing Pipelines

David Andai|Afroglia

2026-05-31

Table of Contents

1	Introduction	2
2	Tabular Data	4
3	Text Data	7
4	Wearable Data	9
5	Image Data	11
6	MRI Data	13
7	Final Takeaway	14

Chapter 1

Introduction

Welcome to brain health data analysis notes. Here, I define brain health data as measurements used to aid the interpretation of brain and mental health-related conditions. These notes focus on the experiences of handling entry-level data issues across various file formats.

We will take a breadth-first approach, covering different approaches to various file formats and highlighting some of the challenges you should be aware of during pre-processing and data harmonisation. The goal is not to become an expert, but to gain an appreciation of the key issues when handling these types of data sets.

The key takeaway here is: brain health data pipelines do not start with analysis or modeling. They start with knowing what kind of data you have (domain knowledge provides an extra advantage), what format it is in, and what pre-processing is required before any analysis can be done.

In these notes, I won't cover databases, deployment, modeling or advanced machine learning workflows. We will cover those in later iterations. The first challenge is understanding what kind of data you have, where it came from, how messy it is, and what needs to be done before any meaningful analysis can begin.

A simple brain health data pre-processing pipeline can be thought of as:

- A -> Data Source
- B -> File Format
- C -> Import Data
- D -> Inspect Structure
- E -> Harmonise Variables
- F -> Preprocess
- G -> Analysis Ready Dataset

Who is this course meant for:

- Organisations/ Teams in brain health
- Medics interested in brain health analysis
- Neuroscience researchers
- Data analyst in brain health

Pre-requisites/ Good to know

- Basic knowledge on Python and/or R
- Background in Data Analysis/Medicine

I will be using common libraries from **Python** and **R** and some packages I often prefer. The code snippets aren't meant to be heavy, so don't stress about them. You are welcome to explore different approaches and share your experience.

Chapter 2

Tabular Data

Tabular data, which is **information organised into structured rows and columns, functions like a spreadsheet** and usually comes in **Comma Separated Values (CSV)** or **Excel files**. This is common in clinical research, data registries and routine health data.

In brain health, this data could include information on participants' demographic characteristics, condition, and other important variables relevant to the topic.

When handed tabular data, the main task is to understand the structure of the data set. The common issues you will find:

- Missing values
- Duplicated records
- Inconsistent naming
- Different data types

For example, one data set may record sex as Male/Female, another as M/F and another as 1/2. Before analysis they need to be harmonised.

```
# Reading CSV file in Python

# Import pandas library
import pandas as pd
import numpy as np

# Replace with your own file path
# use read_csv() to import file
df = pd.read_csv("data/brain_health_sample.csv")
```

```
## OR

# create custom dataset
df = pd.DataFrame({
  "participant_id": [1, 2, 3, 4],
  "sex": ["Male", "F", "Female", "M"],
  "age": [65, 70, np.nan, 59],
  "diagnosis": ["Stroke", "Dementia", "TBI", "Stroke"]
})

# Checking Missingness in Python
df.isna().sum()
```

```
participant_id    0
sex               0
age              1
diagnosis         0
dtype: int64
```

```
# Basic Harmonisation in Python
df["sex_clean"] = df["sex"].replace({
  "M": "Male",
  "F": "Female"
})

df
```

```
   participant_id  sex  age  diagnosis  sex_clean
0                1  Male  65.0    Stroke    Male
1                2    F  70.0  Dementia  Female
2                3 Female   NaN      TBI  Female
3                4    M  59.0    Stroke    Male
```

```
# Reading CSV file in R

# Loads packages quietly
suppressPackageStartupMessages({
  library(rio)
  library(janitor)
  library(tidyverse)
})

# import data using rio
df_r <- import("data/brain_health_sample.csv")
```

```
# OR
# Create demo data

df_r <- tibble(
  PARTICIPANT_ID = c(1, 2, 3, 4),
  Sex = c("Male", "F", "Female", "M"),
  Age = c(65, 70, NA, 59),
  diagnosis = c("Stroke", "Dementia", "TBI", "Stroke")
)

# Clean column names
df_r <- df_r %>%
  clean_names()
df_r
```

```
# A tibble: 4 x 4
  participant_id sex      age diagnosis
      <dbl> <chr>   <dbl> <chr>
1             1 Male     65 Stroke
2             2 F       70 Dementia
3             3 Female   NA TBI
4             4 M       59 Stroke
```

```
# Harmonise variables
df_r <- df_r %>%
  mutate(
    sex_clean = case_when(
      sex == "M" ~ "Male",
      sex == "F" ~ "Female",
      TRUE ~ sex
    )
  )
df_r
```

```
# A tibble: 4 x 5
  participant_id sex      age diagnosis sex_clean
      <dbl> <chr>   <dbl> <chr>      <chr>
1             1 Male     65 Stroke   Male
2             2 F       70 Dementia Female
3             3 Female   NA TBI     Female
4             4 M       59 Stroke   Male
```

Chapter 3

Text Data

Text data comes from **PDFs, Word documents, reports, clinical notes, policy documents, and study forms**. I think from those examples, you get the sense that this is unstructured data. They can also be packaged in tabular form, especially when received as exported CSV or XLSX files.

Currently, large language models can help with text extraction, keyword search, semantic search, and classification. But you really have to know what you are doing. At the end of the day, LLMs are tools, and you have to use them appropriately to get the desired output.

In addition, it is important to distinguish **text analysis** from **qualitative analysis**:

- Text analysis is a specific technique that uses natural language processing (NLP) and machine learning algorithms to systematically parse, extract, and quantify data from written sources.
- Qualitative analysis is a research methodology that interprets non-numerical, text-based data (such as interview transcripts or focus group discussions) to understand underlying meanings and experiences.

In the context of the former (which I like), the pipeline might focus on the following:

- Classifying documents
- Linking text to structured data
- Supporting evidence synthesis

For example, a PDF report may be used to extract information about dementia risk factors, brain health policy priorities or service availability.

```
# Example in python

# create a text df
text_notes = [
    "Participant reports poor sleep and reduced physical activity.",
    "History of stroke with persistent weakness.",
    "Caregiver reports memory decline over the past year."
]

text_df = pd.DataFrame({"note": text_notes})
text_df
```

```

              note
0  Participant reports poor sleep and reduced phy...
1      History of stroke with persistent weakness.
2  Caregiver reports memory decline over the past...
```

```
# Keyword Flagging
text_df["mentions_sleep"] = text_df["note"].str.contains("sleep", case=False)
text_df["mentions_stroke"] = text_df["note"].str.contains("stroke", case=False)

text_df
```

```

              note  ...  mentions_stroke
0  Participant reports poor sleep and reduced phy...  ...      False
1      History of stroke with persistent weakness.  ...      True
2  Caregiver reports memory decline over the past...  ...      False
```

```
[3 rows x 3 columns]
```

Chapter 4

Wearable Data

Wearable data is used to track and analyse personal health and fitness information. This type of data is useful for brain health because it may provide signals related to lifestyle, physical activity, sleep and recovery that affect brain and behavioural patterns.

For analysis, data usually comes in as CSV files or as exported data batches, depending on the device or provider. Examples include:

- Sleep duration
- Movement patterns
- Stress or recovery scores

However, wearable data comes with major challenges:

- High missingness
- Need for domain expertise
- Difficulty distinguishing signal from noise

```
# Example in python
wearable_df = pd.DataFrame({
    "participant_id": [1, 1, 1, 2, 2, 2],
    "date": pd.to_datetime([
        "2026-01-01", "2026-01-02", "2026-01-03",
        "2026-01-01", "2026-01-02", "2026-01-03"
    ]),
    "steps": [4500, np.nan, 6200, 3000, 2800, np.nan],
    "sleep_hours": [6.5, 5.8, np.nan, 7.1, np.nan, 6.9]
})

wearable_df
```

	participant_id	date	steps	sleep_hours
0	1	2026-01-01	4500.0	6.5
1	1	2026-01-02	NaN	5.8
2	1	2026-01-03	6200.0	NaN
3	2	2026-01-01	3000.0	7.1
4	2	2026-01-02	2800.0	NaN
5	2	2026-01-03	NaN	6.9

```
# Missingness Check
wearable_df.isna().sum()
```

```
participant_id    0
date              0
steps            2
sleep_hours       2
dtype: int64
```

```
#Aggregating Wearable Data
wearable_summary = wearable_df.groupby("participant_id").agg(
    mean_steps=("steps", "mean"),
    mean_sleep=("sleep_hours", "mean"),
    days_recorded=("date", "count")
).reset_index()

wearable_summary
```

	participant_id	mean_steps	mean_sleep	days_recorded
0	1	5350.0	6.15	3
1	2	2900.0	7.00	3

Chapter 5

Image Data

Image data is a key component of brain health data pipelines, particularly when working with computer vision, clinical imaging, digital pathology, retinal imaging, neuroimaging slices, or screenshots extracted from reports.

Unlike tabular data, image data is not naturally arranged in rows and columns. An image must first be converted into a numerical format before a computer can analyse it.

Before analysis, image quality should be checked. Image data is not automatically analysis-ready. Before computer vision or machine learning, images must be loaded, resized, normalised, linked to metadata, checked for quality, and organised to avoid data leakage.

Suppose we are building a brain health image dataset from converted MRI slices.

The pipeline may look like this:

- Resize images to a standard size
- Convert images into arrays
- Normalise pixel values
- Create embeddings or model inputs

```
# Example in python
from PIL import Image
import numpy as np

image = Image.open("image/mri_slice.jpg").convert("L")
image = image.resize((128, 128))
image_array = np.array(image) / 255.0 # image_array.shape

image_array
```

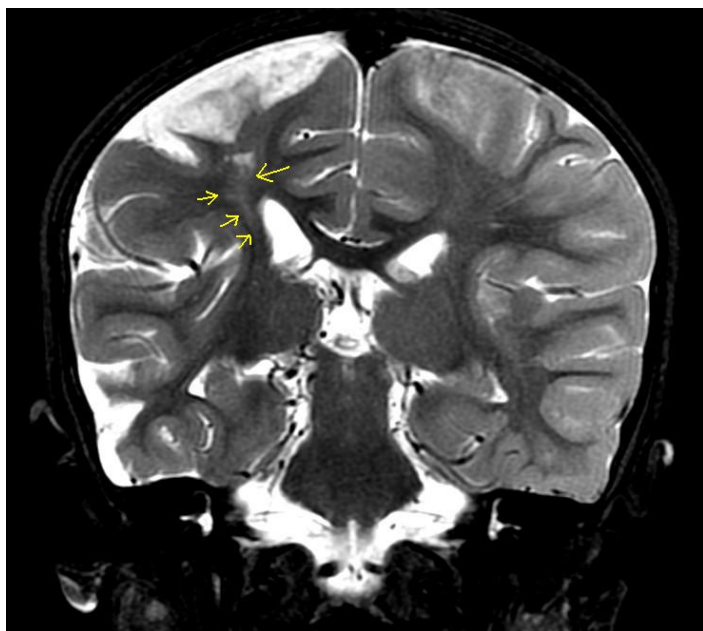


Figure 5.1: Coronal T2-weighted JPG MRI slice showing Wallerian degeneration. source: <https://doi.org/10.53347/rID-18998>

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

Chapter 6

MRI Data

MRI data is more complex than standard image data. MRI files can be in different formats, including **DICOM** and **NIfTI**. In many workflows, DICOM files are converted to NIfTI before analysis. I won't cover the pre-processing steps in these notes. However, here are some of the challenges MRI data presents:

- Different scanner protocols
- Need for anatomical orientation checks
- Operating system differences
- Heavy pre-processing requirements
- Variability across Linux, Windows, and macOS environments

In neuroimaging, tools such as **FreeSurfer** are commonly used because they provide structured outputs, including cortical reconstructions, brain segmentations, and anatomical measurements. FreeSurfer-like outputs are useful for converting raw MRI data into more interpretable features, such as brain region volumes, cortical thickness, and segmentation maps. MRI preprocessing often requires careful environment setup, quality control, and an understanding of the assumptions underlying each tool.

Chapter 7

Final Takeaway

Before modelling brain health data, the first task is to understand the data source. Different data types require different pre-processing strategies. The same logic cannot be applied blindly across tabular, text, wearable, image and MRI data.

Some of the data sets we haven't captured:

- Bioinformatics data
- Signal data

The central message is:

Brain health data work starts with data structure, not algorithms.

This resource is part of **Afroglia's Brain Health Data Notes** series. Afroglia is a brain health platform that translates neuroscience, public health and data science into practical resources for workplace in organisations and research policy-facing teams.

The aim is simple: to make brain health data easier to structure, analyse, interpret, and communicate across Africa.

afroglia.com